

What changed

Primnox V2 — every executed change, before and after. Twenty-six of them, A to Z.

Read it as three columns: what was wrong, what it does now, and the line of code that carries the change. Everything here is in the working tree today — nothing on these pages is planned, partial or intended.

What	Before	After	The line that did it
A One file held the whole app	Everything — layout, sockets, drag-and-drop, every panel — lived in one App.tsx . 1,568 lines at the last commit. src/components/ was an empty directory, so every change touched the same file.	24 component files. App.tsx is shell only now — state, socket, routing — and is down to 901 lines.	<code>src/components/*.tsx 24 files App.tsx 1568 -> 901 lines</code>
B Nine themes nobody could reach	Ten themes were defined in themes.css and applied from a data-theme attribute. Nothing in the app ever set that attribute, so nine of them had never once been rendered.	A picker writes the attribute and remembers the choice. Applied before the first paint, so a light theme no longer flashes black on launch.	<code>initTheme() // main.tsx, before render documentElement.dataset.theme = name</code>
C Four sidebars, four modals	A conversation list, a context panel, and Settings / Knowledge / Memory as fixed inset-0 modals that blanked the whole app behind them.	One 64px icon rail plus one contextual sidebar. Settings, Knowledge and Memory became sections you navigate to. Only the file preview is still an overlay — it is a glance at one file.	<code>const [section, setSection] = useState('chat') // was showGraph/showMemory/showSettings</code>
D Glass that was not glass	The .glass-panel class filled with --nav-bg , which is rgba(7,7,7,0.85) — 85% opaque. Nothing showed through it. The blur was real and invisible. Three orb classes were written and never used.	A separate .px-glass at 12% fill over the blur, mixed from the surface token so it inverts on light themes instead of washing out. The orbs sit behind the transcript, so there is now something to see through.	<code>.px-glass { background: color-mix(in srgb, var(--color-surface) 12%, transparent); backdrop-filter: blur(16px); }</code>
E A sidebar that could not close	The conversation list was always there, and below 768px it covered the chat with no way to dismiss it.	It retracts, remembers the state, and reopens from a button that is always visible. Below 768px it is a drawer with a scrim.	<code>localStorage 'primnox2.chats' chatsOpen governs at every width</code>
F A spinner that never span	The standard reduced-motion reset sets animation-iteration-count: 1 . That does not slow a spinner, it freezes it after one instant rotation. On any machine with Reduce Motion on, every spinner in the app was a static circle — and a static spinner does not read as calm, it reads as HUNG.	Spinners keep turning under reduce, just calmer: 2.4s instead of 1s. WCAG's own carve-out is for motion that carries information, and the one moment the app most needs to show it is alive is this one.	<code>@media (prefers-reduced-motion: reduce) { .px-spin { animation: px-spin 2.4s linear infinite !important; } }</code>

G	No way to copy an answer	You could select text by hand. There was no copy control anywhere — not on replies, not on code blocks.	A copy button on every finished reply and every code block. It appears only once the reply is done, and it reports Blocked if the browser refuses the clipboard rather than silently doing nothing.	<pre><CopyButton text={turn.assistantText} /> // hidden while live - handing over // half a sentence is worse than none</pre>
H	Focus hidden under the composer	The composer sat over the transcript. Tabbing to anything near the bottom scrolled it under the composer — a WCAG 2.2 failure, and just annoying.	The scroller reserves the composer's height, so a focused control is never covered by it.	<pre>pb-48 [scroll-padding-bottom:12rem]</pre>
I	Working and wedged looked the same	A spinner turns at the same rate whether tokens are arriving, the provider has hung, or the socket dropped. There was no way to tell them apart.	A seconds counter beside it. A number can only increment if the turn is genuinely still open. It sits outside the aria-live region — announcing a new number every second would make a screen reader unusable.	<pre>useElapsed(turn.createdAt, live)</pre>
J	Twenty-seven knobs, no dial	GET/PATCH /tunables was finished and complete. The word 'tunable' did not appear anywhere in the frontend.	A panel grouped by key prefix. A knob set from an environment variable renders read-only with its variable name, rather than as a control that silently does nothing.	<pre>step = row.type === 'int' ? 1 : 0.05 // read from the row, never inferred</pre>
K	It guessed instead of asking	Faced with something ambiguous, the model picked one reading and carried on. A guess from a small model does not arrive labelled as a guess — it arrives as a confident sentence, and nothing downstream can tell it apart from an instruction.	An ask_user tool that parks the turn and waits. It cannot auto-approve: unlike a permission prompt there is no safe default answer, because there is no default question.	<pre>broker.ask(question, options) # never auto-approves turn -> awaiting_input</pre>
L	PowerPoint opened as nothing	A .pptx preview showed text with no positions — no layout, no images, no reading order.	Slides render at their real coordinates, with images, a filmstrip and keyboard navigation. Arrow keys are ignored while you are typing in a field, so they never steal a keystroke.	<pre>def _emu_box(shape): # EMU -> fraction return (shape.left / W, shape.top / H, ...)</pre>
M	HTML decks shown as source code	.html was listed under TEXTUAL , so a generated slide deck opened as its own markup.	Rendered in an iframe sandboxed with allow-scripts and deliberately WITHOUT allow-same-origin — with both, the page could reach back into the app's own origin.	<pre>WEB = {".html", ".htm"} sandbox="allow-scripts" # and NOT allow-same-origin</pre>

N	Every chat was called New Chat	<code>rename_conversation</code> existed, but only the manual endpoint ever called it. Nothing auto-titled. Every conversation in the sidebar carried the same name, forever.	Titled from the first message. Derived, not generated — asking the model costs a round trip before the user sees a single token, for a cosmetic string. Guarded on the title still being the default, so a rename you typed is never overwritten.	<pre>def maybe_autotitle(conversation_id, text): # only while title is still default</pre>
O	The WebSocket let anyone in	The check read if origin is not None and origin not in ALLOWED . A browser always sends the header, so a bad origin was refused — but any script that simply omitted it got a 101 and the full event stream. Measured: evil origin 403, no header connected.	Missing is treated as wrong. Measured again after the change: both refused. The test now starts its own server, because the old one had been passing against a stale build (see U).	<pre>if origin not in ALLOWED_ORIGINS: return 403 # None is not in the set</pre>
P	Memory could store an essay	Context injects saved memory whole, on the assumption that it is 'small by construction'. Nothing enforced that. One long memory could crowd out the conversation it was meant to inform.	A cap of 240 characters, as a tunable. It refuses rather than truncating — cutting a sentence in half can reverse its meaning, and a reversed memory is worse than no memory at all.	<pre>raise MemoryTooLong(...) # not text[:240]</pre>
Q	Windows paths arrived mangled	Escapes were decoded by a run of <code>str.replace</code> calls, each reading the last one's output. Newlines were decoded before backslashes, so a path containing <code>lt</code> had the tab pass fire inside it, and the sandbox received a literal TAB where a separator belonged.	One left-to-right pass that consumes both characters of an escape together, which makes the order irrelevant. An unknown escape is left verbatim — <code>ld</code> in a regex has to stay <code>ld</code> , not become a letter d.	<pre>while i < len(value): if ch == '\\' and nxt in _ESCAPES: out.append(_ESCAPES[nxt]); i += 2</pre>
R	A misspelled tool just failed	Call <code>read_files</code> instead of <code>read_file</code> and the step died with 'unknown tool'. The model would usually try the same wrong name again.	The error names the closest real tools, so the retry has somewhere to go.	<pre>difflib.get_close_matches(name, real, n=3, cutoff=0.6)</pre>
S	It claimed work it had not done	Nothing in the prompt forbade inventing a filename, an asset id, or a whole action. On a 7B those are the normal failures, not the rare ones.	Four rules, each aimed at a failure with an observable signature — a fabricated id, a claim with no tool call behind it. Kept to four because every line is charged to every turn forever.	<pre>"Do not invent:" "- Tool names, file names, asset ids..." "- Work you did not do..."</pre>
T	A test that hung for 20 minutes	<code>def ask(..., timeout_s = DEFAULT_TIMEOUT_S)</code> — a default argument binds once, at import. Changing the tunable afterwards changed nothing, so the test sat waiting out the original timeout.	Read at call time. Measured: 1200.41s before, 2.21s after.	<pre>def ask(..., timeout_s: int None = None): timeout_s = timeout_s or _tune(...)</pre>

U	A test pointed at the wrong build	The WebSocket origin test connected to the server already running on port 4109 — which was the unpatched build. It passed against code that still had the bug in it.	It starts its own server on an ephemeral port. Proven by reverting the fix and watching the test fail.	port = 0 # its own server, # on an ephemeral port
V	Synthetic memories signed as yours	The benchmark pack mapped confidence onto provenance: stated became explicit , which the Memory list renders as you said . Twenty-six invented sentences were shown to you as your own words, on the one screen whose whole job is to show what the system believes about you.	Every imported row is imported . Provenance answers 'where did this come from', and for all of them the honest answer is 'a synthetic pack'.	PROVENANCE = {c: 'imported' for c in CONFIDENCES}
W	The seeder wrote to your real DB	<code>python sdl/inject.py --pack memory-100</code> — the shortest thing anyone would type, and the exact line in the file's own docstring — defaulted to the app's live store. It happened. All 100 memories you had were synthetic and the real count was zero.	There is no default any more. <code>--db</code> points anywhere; <code>--app-db</code> names the real store and costs a deliberate sentence. Nothing was mistyped last time, because nothing was typed — that is the part that got fixed.	if not args.db and not args.app_db: raise SystemExit('no destination...')
X	Asking about you searched the code	Asked what do I prefer , the model called graph_query , got 24 lines of Primnox's own source back, and answered from them. Nothing underneath was broken — these are separate tables and separate tools. It had no sentence telling it which question goes where.	Each tool now names the other. And when a first-person question reaches the graph anyway, the result is prefixed with a warning — before the citations, not after, because 2,000 tokens of graph read first is an answer already forming.	description="...It holds nothing about the user: for what they prefer, use recall_memory instead."
Y	The app shifted up 50px and stuck	The shell used overflow-hidden , which makes an element a scroll container. Focusing the composer scrolled it — and with no scrollbar, there was no way to scroll back.	overflow-clip clips without becoming scrollable.	- overflow-hidden + overflow-clip
Z	A modal that mounted invisible	The asset viewer was a motion element inside AnimatePresence . It mounted with its initial styles — opacity 0 — and no animation ever started. It was in the DOM, readable to a script, and invisible to a person.	Plain CSS entry, with nothing to go wrong. A modal that silently fails to appear is a worse defect than a modal without a fade.	// deliberately not a motion element

Where that leaves it

Tests	The suite grew alongside every fix on the previous pages.	535 passing , measured just now. 22 of those were written today, for W and X alone. Every defect listed here that could be tested has a test that fails without its fix.
Still open	The HTTP API accepts any request from any local process, with no token at all. Anything running on this machine can read every conversation.	Not fixed. It needs a token scheme on both ends and a decision about what breaks. That is yours to make, not something to change quietly.
Left alone on purpose	<code>context.history_turns</code> is 2000, and every turn fills the token budget.	This is working as designed: the budget trims history and the prompt never exceeds it. Lowering the cap was considered and rejected — at 100 it caused a critical finding where conversations silently lost the turns a user cites by number. The cost is real, but the cure was worse.

Nine of the twenty-six were found by using the app rather than by reading it — the glass, the spinner, the sidebar, the 50px shift. Four (O, T, U, W) were visible only from a measurement and would have gone on passing review indefinitely. Two of those four were defects in the tests themselves.